
开 始
文件包含
#include <iostream>
#include <fstream>
#include <sstream>
#include <ctime>
#include <vector>
#include <string>
#include <assert.h>
#include <pcl/io/pcd_io.h>
#include <pcl/point_types.h>
#include <pcl/filters/filter_indices.h>
#include <pcl/filters/statistical_outlier_removal.h>
#include <pcl/filters/extract_indices.h>
#include <pcl/kdtree/kdtree_flann.h>
#include <pcl/ModelCoefficients.h>
#include <pcl/filters/project_inliers.h>
#include <pcl/segmentation/sac_segmentation.h>
#include <pcl/sample_consensus/method_types.h>
#include <pcl/sample_consensus/model_types.h>
#include <pcl/sample_consensus/ransac.h>
#include <pcl/correspondence.h>
#include <pcl/registration/correspondence_estimation.h>
#include <pcl/registration/transformation_estimation_svd.h>
#include <pcl/common/centroid.h>
#include <boost/shared_ptr.hpp>
#include "DBSCAN_simple.h"
#include "DBSCAN_precomp.h"
#include "DBSCAN_kdtree.h"
函数返回值定义开始
创建枚举类型返回值
创建返回值变量 enum CalibrationReturnNum{ };
定义数据读入错误码
输入读入错误码定义 DATA_READING_ERROR = 0,
定义模型拟合未完成错误码
模型拟合未完成错误码定义 MODEL_FITTING_NOT_COMPLETED = 1,
定义数据处理未完成错误码
数据处理未完成未完成错误码定义 DATA_PROCESSING_NOT_COMPLETED = 2,
定义程序运行成功编码
程序运行成功编码定义 OK = 3
函数返回值定义结束
平面拟合函数定义开始
函数名、参数定义
void fitPlaneUsingSVD(pcl::PointCloud<pcl::PointXYZ>::Ptr& pointcloud, pcl::ModelCoefficients::Ptr coefficients){ }
创建质心存储变量
质心存储变量创建 Eigen::Matrix<double, 4, 1> centroid;
计算点云质心
质心计算 pcl::compute3DCentroid(*pointcloud, centroid);
创建协方差矩阵存储变量
协方差矩阵变量创建 Eigen::Matrix<double, 3, 3> convariance_matrix;
计算点云协方差
协方差计算 pcl::computeCovarianceMatrix(*pointcloud, centroid, convariance_matrix);
创建特征向量存储变量
特征向量变量创建 Eigen::Matrix3d eigenVectors;
创建特征值存储变量
特征值变量创建 Eigen::Vector3d eigenValues;
计算特征值特、征向量
特征值、特征向量计算 pcl::eigen33(convariance_matrix, eigenVectors, eigenValues);
创建最小特征值位置存储变量
最小特征值位置变量创建 Eigen::Vector3d::Index minRow, minCol;
寻找最小特征值
最小特征值位置查找 eigenValues.minCoeff(&minRow, &minCol);
计算平面方程系数
平面方程系数计算
double A = eigenVectors(0, minRow);
double B = eigenVectors(1, minRow);
double C = eigenVectors(2, minRow);
double D = -(A*centroid(0)+B*centroid(1)+C*centroid(2));
保存平面方程系数
平面系数保存
coefficients->values.resize(4);
coefficients->values[0] = static_cast<float>(A);
coefficients->values[1] = static_cast<float>(B);
coefficients->values[2] = static_cast<float>(C);
coefficients->values[3] = static_cast<float>(D);
平面拟合函数定义结束
旋转矩阵计算函数定义开始
定义函数名、参数等
void computeTransformMatrix(pcl::ModelCoefficients::Ptr coefficients_before, pcl::ModelCoefficients::Ptr coefficients_after, Eigen::Matrix4f& transform_matrix){ }
创建旋转前向量、旋转后向量、旋转轴变量
变量创建 Eigen::Vector3f normal_before, normal_after, rotation_axis;
旋转前变量赋值
变量赋值
normal_before[0] = coefficients_before->values[0];
normal_before[1] = coefficients_before->values[1];
normal_before[2] = coefficients_before->values[2];
旋转后变量赋值
变量赋值
normal_after[0] = coefficients_after->values[0];
normal_after[1] = coefficients_after->values[1];
normal_after[2] = coefficients_after->values[2];
计算旋转轴
旋转轴计算 rotation_axis = normal_before.cross(normal_after);
旋转轴单位化
向量单位化 rotation_axis = rotation_axis.normalized();
计算旋转角
旋转角计算 double theta = acos(normal_before.dot(normal_after) / (normal_before.norm() * normal_after.norm()));
创建旋转矩阵计算临时变量
临时变量计算 double a = cos(0.5 * theta), b = sin(0.5 * theta) * rotation_axis[0], c = sin(0.5 * theta) * rotation_axis[1], d = sin(0.5 * theta) * rotation_axis[2];
初始化旋转矩阵变量
矩阵初始化为单位阵 transform_matrix = Eigen::Matrix4f::Identity();
计算旋转矩阵
旋转矩阵计算
transform_matrix(0, 0) = 1 - 2 * c * c - 2 * d * d;
transform_matrix(0, 1) = 2 * b * c - 2 * a * d;
transform_matrix(0, 2) = 2 * a * c + 2 * b * d;
transform_matrix(1, 0) = 2 * b * c + 2 * a * d;
transform_matrix(1, 1) = 1 - 2 * b * b - 2 * d * d;
transform_matrix(1, 2) = 2 * c * d - 2 * a * b;
transform_matrix(2, 0) = 2 * b * d - 2 * a * c;
transform_matrix(2, 1) = 2 * a * b + 2 * c * d;
transform_matrix(2, 2) = 1 - 2 * b * b - 2 * c * c;
旋转矩阵计算函数定义结束
点云排序函数定义开始
定义函数名、参数等
函数头定义 bool comparePoint(pcl::PointXYZ pt1, pcl::PointXYZ pt2){ }
判断条件
条件判断 if (pt1.x != pt2.x)
是
条件为真，返回
返回 return pt1.x < pt2.x;
点云排序函数定义结束
矩阵保存函数定义开始
定义函数名、参数等
函数头定义 void saveTransformMatrix(Eigen::Matrix4f trans_mat,char* trans_txt){ }
创建输出对象变量
输出对象变量创建 std::ofstream trans_save;
打开输出文本
文本打开 trans_save.open(trans_txt, std::ios::out | std::ios::trunc);
循环1开始
循环1 for (size_t i = 0; i < 4; ++i)
循环2(循环1内)开始
循环2 for (size_t j = 0; j < 4; ++j)
写数据到文本文件
写数据 trans_save << trans_mat(i, j);
否
判断条件
条件判断 if (j != 3)
是
写逗号分隔符
写逗号 trans_save << ",";
写换行符
写换行符 trans_save << std::endl;
循环2结束
循环1结束
关闭文本文件
关闭文本，trans_save.close();
矩阵保存函数定义结束
标定函数定义开始
定义函数名、参数等
函数头定义 int calibration(char* cal_data1, char* cal_data2, int cal_sor_k, double cal_sor_s, double cal_ellipse_dis, int cal_ellipse_iterations, double& cal_avg_dis, char* cal_transform_matrix){ }
创建模型中心存储变量
变量创建 std::vector<pcl::PointCloud<pcl::PointXYZ>::Ptr, Eigen::aligned_allocator<pcl::PointCloud<pcl::PointXYZ>::Ptr> > model_center;
创建标定块点云存储变量
变量创建 std::vector<std::string> data_in(2);
保存标定块点云
数据保存
data_in.push_back(cal_data1);
data_in.push_back(cal_data2);
循环3开始
循环3 for (size_t i = 0; i < 2; ++i)
创建点云存储变量
变量创建
pcl::PointCloud<pcl::PointXYZ>::Ptr pointcloud_in(new pcl::PointCloud<pcl::PointXYZ>);
pcl::PointCloud<pcl::PointXYZ>::Ptr pointcloud_ellipse_center(new pcl::PointCloud<pcl::PointXYZ>);
否
条件判断
判断条件 if (pcl::io::loadPCDFile(data_in[i + 2], *pointcloud_in) == -1)
是
返回
返回错误编码
return CalibrationReturnNum::DATA_READING_ERROR;
创建滤波点云存储变量
变量创建 pcl::PointCloud<pcl::PointXYZ>::Ptr pointcloud_filtered(new pcl::PointCloud<pcl::PointXYZ>);
创建统计滤波类对象
统计滤波类对象 pcl::StatisticalOutlierRemoval<pcl::PointXYZ> sor;
输入点云
点云输入 sor.setInputCloud(pointcloud_in);
设置临近点个数
临近点个数设置 sor.setMeanK(cal_sor_k);
设置标准差倍数
标准差倍数设置 sor.setStddevMulThresh(cal_sor_s);
滤波
滤波计算 sor.filter(*pointcloud_filtered);
创建平面系数存储变量
变量创建 pcl::ModelCoefficients::Ptr coefficients_plane(new pcl::ModelCoefficients);
拟合平面
调用函数进行平面拟合 fitPlaneUsingSVD(pointcloud_filtered, coefficients_plane);
创建投影点云保存变量
变量创建 pcl::PointCloud<pcl::PointXYZ>::Ptr pointcloud_proj(new pcl::PointCloud<pcl::PointXYZ>);
创建点云投影类对象
点云投影类对象创建 pcl::ProjectInliers<pcl::PointXYZ> proj;
设置模型
设置平面模型 proj.setModelType(pcl::SACMODEL_PLANE);
输入点云
点云输入 proj.setInputCloud(pointcloud_filtered);
输入模型系数
模型系数输入 proj.setModelCoefficients(coefficients_plane);
点云投影
投影计算 proj.filter(*pointcloud_proj);
创建Z轴系数存储变量
变量创建 pcl::ModelCoefficients::Ptr coefficients_Z(new pcl::ModelCoefficients);
Z轴系数变量赋值
变量赋值
coefficients_Z->values.resize(3);
coefficients_Z->values[0] = 0;
coefficients_Z->values[1] = 0;
coefficients_Z->values[2] = 1;
创建旋转矩阵保存变量
变量创建 Eigen::Matrix4f transform_matrix = Eigen::Matrix4f::Identity();
计算旋转矩阵
旋转矩阵计算 computeTransformMatrix(coefficients_plane, coefficients_Z, transform_matrix);
创建旋转点云保存变量
变量创建 pcl::PointCloud<pcl::PointXYZ>::Ptr pointcloud_proj_rot(new pcl::PointCloud<pcl::PointXYZ>);
旋转点云
点云旋转 pcl::transformPointCloud(*pointcloud_proj, *pointcloud_proj_rot, transform_matrix);
计算旋转矩阵的逆矩阵
逆矩阵计算 Eigen::Matrix4f transform_matrix_inverse = transform_matrix.inverse();
创建边界点云存储变量
变量创建 pcl::PointCloud<pcl::PointXYZ>::Ptr pointcloud_edge(new pcl::PointCloud<pcl::PointXYZ>);
创建半径及临近点个数存储变量
double radius = 0.1;
int numofNeighbor = 40;
判断条件
if(std::fabs(i - 1) < 0.00000001)
{
radius = 0.2;
numofNeighbor = 22;
}
创建kd树
pcl::KdTreeFLANN<pcl::PointXYZ> kdtree;
输入 点云
kdtree.setInputCloud(pointcloud_proj_rot);
创建邻域点索引及平方距离存储变量
std::vector<int> pointIdxRadiusSearch;
std::vector<float> pointRadiusSquaredDistance;
根据当前点邻域点个数进行边界点搜索
for (size_t i = 0; i < pointcloud_proj_rot->points.size(); ++i)
{
if (kdtree.radiusSearch(pointcloud_proj_rot->points[i], radius, pointIdxRadiusSearch, pointRadiusSquaredDistance) > 0)
{
if (pointIdxRadiusSearch.size() < numofNeighbor)
{
pointcloud_edge->points.push_back(pointcloud_proj_rot->points[i]);
}
}
}
创建高度值保存变量并赋值
高度存储 float z_value = pointcloud_edge->points[0].z;
创建kd树对象
kd树创建 pcl::search::KdTree<pcl::PointXYZ>::Ptr tree(new pcl::search::KdTree<pcl::PointXYZ>);
构建kd树
kd树构建 tree->setInputCloud(pointcloud_edge);
创建聚类索引保存变量
变量创建 std::vector<pcl::PointIndices> cluster_indices;
创建密度聚类类对象
密度聚类对象创建 DBSCANKdtreeCluster<pcl::PointXYZ> ec;
设置最少核心点数
最少核心点数设置 ec.setCorePointMinPts(5);
设置聚类距离阈值
距离阈值设置 ec.setClusterTolerance(0.8);
设置最小类点数
最小类点数设置 ec.setMinClusterSize(20);
设置最大类点数
最大类点数设置 ec.setMaxClusterSize(2000);
设置搜索方式
ec.setSearchMethod(tree);
输入点云
设置输入点云 ec.setInputCloud(pointcloud_edge);
聚类
聚类计算 ec.extract(cluster_indices);
循环4（循环3内）开始
for (std::vector<pcl::PointIndices>::const_iterator it = cluster_indices.begin(); it != cluster_indices.end(); ++it)
创建临时变量
变量创建
pcl::PointCloud<pcl::PointXYZ>::Ptr cloudtemp(new pcl::PointCloud<pcl::PointXYZ>);
pcl::PointXYZ pointtemp;
循环5（循环4内）开始
循环开始 for (std::vector<int>::const_iterator pit = it->indices.begin(); pit != it->indices.end(); ++pit)
点坐标存储变量赋值
变量赋值
pointtemp.x = pointcloud_edge->points[*pit].x;
pointtemp.y = pointcloud_edge->points[*pit].y;
pointtemp.z = pointcloud_edge->points[*pit].z;
点保存
点保存 cloudtemp->points.push_back(pointtemp);
循环5结束
设置点云宽度、高度
点云尺寸设置
cloudtemp->width = (uint32_t)cloudtemp->points.size();
cloudtemp->height = 1;
创建模型系数存储变量
pcl::ModelCoefficients::Ptr coefficients_model(new pcl::ModelCoefficients);
创建模型内点存储变量
pcl::PointIndices::Ptr inliers(new pcl::PointIndices);
创建模型拟合类对象
pcl::SACSegmentation<pcl::PointXYZ> seg_model;
设置系数优化
seg_model.setOptimizeCoefficients(true);
设置模型
seg_model.setModelType(pcl::SACMODEL_ELLIPSE2D);
设置方法
seg_model.setMethodType(pcl::SAC_RANSAC);
设置距离阈值
seg_model.setDistanceThreshold(cal_ellipse_dis);
设置最大迭代次数
seg_model.setMaxIterations(cal_ellipse_iterations);
输入点云
seg_model.setInputCloud(cloudtemp);
计算模型系数
seg_model.segment(*inliers, *coefficients_model);
否
判断条件
条件判断 if (coefficients_model->values[2] / coefficients_model->values[3] < 3 && coefficients_model->values[2] < 8)
是
创建点存储变量
变量创建 pcl::PointXYZ ct;
点赋值
变量赋值
ct.x = coefficients_model->values[0];
ct.y = coefficients_model->values[1];
ct.z = z_value;
点保存
点保存 pointcloud_ellipse_center->points.push_back(ct);
循环4结束
否
条件判断
判断条件 if pointcloud_ellipse_center->points.size() != 3)
是
返回
返回错误码 return CalibrationReturnNum::MODEL_FITTING_NOT_COMPLETEDD;
设置点云尺寸
点云尺寸设置
pointcloud_ellipse_center->width = (uint32_t)pointcloud_ellipse_center->points.size();
pointcloud_ellipse_center->height = 1;
旋转点云
点云旋转 pcl::transformPointCloud(*pointcloud_ellipse_center, *pointcloud_ellipse_center, transform_matrix_inverse);
点云排序
点排序 std::sort(pointcloud_ellipse_center->begin(),pointcloud_ellipse_center->end(), comparePoint);
保存模型中心
模型中心保存 model_center.push_back(pointcloud_ellipse_center);
循环3结束
否
条件判断
条件判断 if (model_center.size() != 2)
是
返回
返回错误码 return CalibrationReturnNum::DATA_PROCESSING_NOT_COMPLETED;
创建点云存储变量
变量创建
pcl::PointCloud<pcl::PointXYZ>::Ptr model_center10(new pcl::PointCloud<pcl::PointXYZ>);
pcl::PointCloud<pcl::PointXYZ>::Ptr model_center11(new pcl::PointCloud<pcl::PointXYZ>);
pcl::PointCloud<pcl::PointXYZ>::Ptr model_center2(new pcl::PointCloud<pcl::PointXYZ>);
变量赋值
变量赋值
model_center10 = model_center[0];
model_center2 = model_center[1];
创建y轴转换矩阵变量
变量创建并初始化为单位阵 Eigen::Matrix4f transform_y_axis = Eigen::Matrix4f::Identity();
旋转矩阵赋值
矩阵赋值 transform_y_axis(1, 1) = -1;
指定点云宽高
model_center10->width = (uint32_t)model_center10->points.size();
model_center10->height = 1;
旋转点云
旋转 pcl::transformPointCloud(*model_center10, *model_center11, transform_y_axis);
创建对应点对存储变量
变量创建 boost::shared_ptr<pcl::Correspondences> corr(new pcl::Correspondences);
循环6开始
循环开始 for (size_t i = 0; i < 3; ++i)
创建临时变量
变量创建 pcl::Correspondence cr;
变量赋值
赋值
cr.index_query = i;
cr.index_match = i;
保存
保存 corr->push_back(cr);
循环6结束
创建矩阵变换保存变量
变量创建 Eigen::Matrix4f transform12 = Eigen::Matrix4f::Identity();
创建变换矩阵估计类对象
类对象创建
pcl::registration::TransformationEstimationSVD<pcl::PointXYZ, pcl::PointXYZ, float>::Ptr
trans(new pcl::registration::TransformationEstimationSVD<pcl::PointXYZ, pcl::PointXYZ, float>);
解算变换矩阵
变换矩阵解算 trans->estimateRigidTransformation(*model_center11, *model_center2, *corr, transform12);
计算最终变换矩阵
Eigen::Matrix4f trans_mat = transform12 * transform_y_axis;
保存转换矩阵到本地
矩阵保存 saveTransformMatrix(trans_mat ,cal_transform_matrix);
创建点云存储变量
变量创建 pcl::PointCloud<pcl::PointXYZ>::Ptr center11_trans(new pcl::PointCloud<pcl::PointXYZ>);
旋转点云
旋转 pcl::transformPointCloud(*model_center11, *center11_trans, transform12);
创建临时变量
临时变量创建 double sum = 0;
循环7开始
循环开始 for (size_t i = 0; i < 3; ++i)
计算模型对应点距离累加和
距离累加
sum += sqrt((center11_trans->points[i].x - model_center2->points[i].x) * (center11_trans->points[i].x - model_center2->points[i].x) +
(center11_trans->points[i].y - model_center2->points[i].y) * (center11_trans->points[i].y - model_center2->points[i].y) +
(center11_trans->points[i].z - model_center2->points[i].z) * (center11_trans->points[i].z - model_center2->points[i].z));
循环7结束
计算距离均值
均值计算 cal_avg_dis = sum / 3.0;
返回
返回 return CalibrationReturnNum::OK;
标定函数定义结束
结 束